



A ROBONUGGETS GUIDE

Build your Agentio OS

The complete ARMS setup for the Claude 5 generation - Applications, Routines, Memory, and Skills in three levels each, every starter prompt from the lesson, and the visual command centre that sits on top.



README

What this is. The in-depth setup guide for your agentic operating system. Four parts (ARMS), three levels per part, one copy-paste prompt per level. Everything in this PDF is selectable text, so any prompt can go straight into your agent.

How to use it. Read the framework below, then work bottom-up the way the lesson teaches it: Skills (pages 3-5), Memory (pages 6-7), Routines (pages 8-9), Applications (pages 10-12). Skip straight to the first level you have not built. Finish with the command centre (pages 13-14) and the 7-day plan (page 15).

The shortcut. Inside the RoboNuggets community at skool.com/robonuggets, a pre-built version of this OS - the command centre and the visual second brain - is available to members, who also learn to build and sell AI systems like it for real businesses.

The ARMS framework

An **agent** is an AI that does real work: it reads and writes files, uses tools, and finishes tasks. An **operating system** is the standing structure around it - what it shows you, what it runs on schedule, what it remembers, and what it can reliably do. Get these four right and you are ahead of 99% of agentic AI users. Each part has three levels: level 1 works today, level 3 is where the system starts working without you.

A



Applications

What your agent connects to, and the tools it builds for you. Browse connectors, then search them, then build your own.

R



Routines

Scheduled work. Local first, then always-on with its own machine in the cloud.

M



Memory

Your workspace and context. A folder, then router files, then a visual second brain.

S



Skills

Your SOPs as commands. Pre-built, then file-set skills, then triggered from outside chat.



Skills



Skills are shortcuts to your standard operating procedures. The rule: when you find yourself prompting the same task twice, make it a skill. Your work is custom to you, so the habit of creating your own skills is what turns a generic assistant into your assistant.

L1**Use pre-built skills, then create your own** START HERE

Browse the skills your AI tool ships with (in the Claude desktop app: Customize, then Skills). The one to learn first is the skill-creator skill: see a good tip anywhere - a post, a thread, a video - paste it in and have your agent turn it into a skill you can test the same day.

L2**Skills with their own file sets**

A skill is not just one markdown file. The most powerful ones carry rich references - visual examples, templates, sub-files - with the main skill file acting as a router that points the agent to the right reference for each job. A design skill with a real brand book beats a design skill described in words.

L3**Trigger skills outside of chat**

Headless mode runs a skill without opening a chat window: one command sends a one-shot prompt using the model and effort level you choose. That is how skills become buttons in your own dashboard, or steps inside internal tools for your team.

You're done when: your most repeated task runs from one named command, and your favourite skill fires from a button, not a chat.

LEVEL 1 · YOUR FIRST SKILL

I keep doing the same kind of task with you over and over. Turn it into a skill so I can run it with one command.

The task: [describe it, e.g. "clean my downloads folder and file everything where it belongs"]

Ask me first, in one batch:

1. What should trigger it: a /name, certain phrases, or both?
2. What does a perfect run look like, step by step?
3. What should this skill never do?

Then create the skill folder with a SKILL.md that covers: the name, a description with trigger words, the steps, and the guardrails. Keep it under 60 lines - short skills get followed, long ones get skimmed.

Test it by running the skill once for real, and show me the result.

LEVEL 2 · TURN THICK SKILLS INTO A SKILL TREE

Look through my skills folder (.claude/skills/) and find my "thick" skills - any skill where one SKILL.md file is doing all the work (roughly 150+ lines, or several different jobs mixed into one file).

Before restructuring anything, ask me these in one batch:

1. Which skills do I actually use most weeks?
2. Are any skills off-limits and not to be touched?
3. How aggressive should the split be: light cleanup, or a full file tree per skill?

Then restructure each thick skill into a skill tree:

- SKILL.md becomes a short router: what the skill is, when it triggers, and a table pointing to the right file for each job
- Each distinct job, reference list, or template moves into its own clearly named file inside the skill folder
- The router tells the agent to read ONLY the file that matches the task, never everything at once

Rules: do not change what any skill does, only how it is organized. Keep every trigger and every rule. A good split makes SKILL.md shorter, not longer. Show me the before and after line counts, and wait for my go before touching files.

LEVEL 3 · RUN SKILLS FROM YOUR OWN SOFTWARE

I want to run my Claude Code skills from buttons in my own software, without opening a chat window. This works through headless mode: the command `claude -p "prompt"` runs one prompt and exits.

Before you build anything, ask me these in one batch:

1. Which app should hold the buttons: my dashboard HTML, a local web app, or something else?
2. Which 3-5 skills do I run often enough to deserve a button?
3. Should each button offer options (model, effort level, an extra input field), or fire with defaults?

Then build it:

- Add one button per skill to my app. Each button runs the matching skill headlessly, for example:
`claude -p "/clean-up" --model claude-sonnet-5`
- Buttons that need options get a small picker (model, effort) that feeds into the command
- Show run status next to each button (running, done, failed) and where the output landed
- If my app is plain HTML with no server, add the smallest possible local helper: one endpoint that shells out to `claude -p`, and wire the buttons to it

Log every run to a simple `runs.log` file so I can see what fired and when. Walk me through one real button press, end to end, before calling it done.

WHY HEADLESS MATTERS

A skill you trigger from a button is a skill your whole system can use: dashboards, internal tools, even other routines. No extra technical tooling to learn - the same agent that built the skill wires the button. Run heavier models and higher effort only where the task earns it; a cleanup run does not need your best model.

Memory



The more you work with your agent, the more context and files build up. Memory is how that buildup stays an asset instead of a tax. The symptom to watch for: retrieval slows down, and your token usage climbs because the agent is wading through clutter to find things.

L1**A workspace is just a folder** START HERE

Point your agent at one folder and work there. This is fine at the start - the problem arrives when the files pile up (a real workspace can quietly reach tens of thousands of files) and the agent slows down finding anything.

L2**Router files, not neat folders**

In the old paradigm you renamed and nested folders so YOU could navigate. Agents parse files at lightning speed - what they need is a map, not tidiness. A master router file names your areas of work and points each one to its own index; each index lists that area's key files in one line each. Fastest path, fewest steps.

L3**A visual second brain**

A local page that shows your workspace as a living, searchable map: how files connect, grouped by area, found in seconds. Faster than file explorer, and far better for showing your work to other people - clients, teammates, or an audience.

You're done when: a brand-new session finds the right file through the routers on the first hop, and you can search your whole workspace visually.

LEVEL 2 · BUILD ROUTER FILES

My workspace has grown and it is getting hard for you to find things. Build me router files instead of reorganizing my folders.

A router file is a short markdown index that points an agent to where things live, so it can jump straight to the right folder instead of searching everything.

Ask me first, in one batch:

1. What are the 3-6 main areas of my work (e.g. content, clients, money, health)?
2. Which files or folders do I touch most weeks?
3. Is anything private that should stay out of the indexes?

Then:

1. Explore the workspace and map what actually lives where
2. Write one master router at the root (CLAUDE.md if this is Claude Code): who I am in one line, my main areas, and a table pointing each area to its folder and its own index file
3. Write one short index per area (like CONTENT.md or CLIENTS.md) listing its key files, one line each
4. Keep every index under a page. These are pointers, not documentation
5. End the master router with the rule you will follow from now on: when a file moves or a new project starts, update the router in the same turn - a stale pointer is worse than no pointer

LEVEL 3 · VISUAL SECOND BRAIN

Build me a visual second brain: one local HTML page that shows my workspace as an interactive map, so I can find any file faster than file explorer.

Ask me first, in one batch:

1. Which folders should it cover, and which should it skip?
2. What do I search by most: topic, project name, or date?
3. Dark or light look, and any brand colors to use?

Then build a single self-contained HTML file that:

- Reads an index of my chosen folders and draws the files as a zoomable graph or grid, grouped by folder and topic
- Lets me search as I type, and filter by type or folder
- Shows a preview and the full path with a copy button when I click a file
- Ships with a small script I rerun to refresh the index when files change

Keep it to one HTML file plus the one index file it reads. No frameworks, no internet needed. When it works, tell me the exact path and how to refresh it.



Routines



Routines are scheduled tasks - and in practice, a routine is just a prompt your agent sends itself at a time you set. This layer comes third for a reason: once your skills and memory are solid, you can trust the agent to do work without you watching.

L1**Local routines** START HERE

Draft routines in natural language with whatever your tool ships (in the Claude desktop app: the Routines sidebar). A daily draft, a weekly digest - the output lands where you will see it. When a routine uses one of your custom skills, the result arrives mostly in your voice and only needs minor edits. Limitation: local routines only run while your computer is on.

L2**Always-on, with its own machine**

Give an agent its own computer in the cloud (a small virtual private server) so routines fire 24/7 with your laptop closed. The piece most people miss: that agent needs your skills and context. A free open source tool called Syncthing syncs your workspace between your computer and the always-on machine, so both agents see the same files.

L3**The whole agent in the cloud**

Some builders now install their coding agent directly on the cloud machine, so all context lives there - no syncing at all. Expect the big AI companies to offer this out of the box eventually; file security is the hard part. Until then, level 2 gives you the same result today.

You're done when: at least one piece of real work ships on schedule with your computer off.

LEVEL 1 · YOUR FIRST LOCAL ROUTINE

Set up my first local routine: a task that runs on a schedule while my computer is on, without me asking.

The routine I want: [e.g. "every Friday at 9am, draft my weekly newsletter from this week's notes and leave it in drafts/"]

Ask me first, in one batch:

1. Exactly when should it run, and in what timezone?
2. Where should the output land, and should anything notify me when it is done?
3. If the computer was off at the scheduled time, should it skip that run or fire on next boot?

Then:

1. Write the routine as a script or a scheduled agent prompt
2. Register it with my system's scheduler, or with Claude's own scheduled tasks if this setup has them
3. Run it once right now as a test and show me the output
4. Add a one-line log per run so I can check later that it has been firing

Finish by telling me how to pause it, change its time, and remove it.

LEVEL 2 · TWO-MACHINE SYNC WITH SYNCTHING

You are setting up one half of a two-machine sync. This same prompt is being run on my other machine, so first detect which machine you are on and do your half.

Goal: keep one shared folder in sync between my main computer and my always-on machine using Syncthing, so agents on both sides see the same files.

Before touching anything, ask me these in one batch:

1. Which folder should sync: my whole workspace, or just a shared subfolder?
2. What is the other machine (name and OS), and how do the two machines reach each other: same network, Tailscale, or over the internet?
3. What must NEVER sync (secrets, node_modules, temp folders)?

Then:

1. Install Syncthing for this OS if it is missing, and set it to start on boot
2. Add the folder I chose, with an ignore file covering my exclusions
3. Show me this machine's Device ID, then ask me for the other machine's ID and add it as a trusted device
4. Share the folder with that device and wait for the other side to accept
5. Verify: create a test file, confirm it appears on the other machine, then delete it
6. Report exactly what you set up and how I check sync status later

Security rule: never expose the Syncthing web UI to the internet. Local access only.



Applications



Real work means reaching your real apps - email, calendar, your editing tools, your community platform. Connectors come in three usual formats: a CLI (command-line tool), an API, or an MCP server. Your agent can find, vet, and even build them for you.

L1**Browse what is available** START HERE

Your tool ships with a catalog (in the Claude desktop app: Customize, then Connectors). Connect the obvious ones - email, calendar, drive. Fine to start, but browsing by hand is not the most efficient way in.

L2**Have your agent search**

Instead of you hunting, the agent searches the web for an official connector first, then community-made ones: CLI, API, or MCP. It shortlists candidates, you pick, it scans the code for safety and sets it up. That is how an app with no official integration - a video editor, a niche platform - still ends up wired to your agent.

L3**Build your own connectors and apps**

When nothing exists, build it: a small CLI for the platform that has none, or a micro-app for a job you do weekly. A dashboard, a generations gallery, a diagram landing pad - each one is a single prompt away once the rest of your OS is in place. This is where you feel how powerful these platforms really are.

You're done when: the apps you live in are reachable by your agent, and one custom tool you built yourself is part of your week.

LEVEL 1 · CONNECTOR AUDIT

Audit my connectors: look at what I actually do with you, then tell me what is worth plugging in.

1. List the connectors and MCP servers available to you in this setup, and mark which are already connected and working
2. Ask me in one batch:
 - What are the 5 apps I live in every week (email, calendar, notes, business tools)?
 - Which tasks do I hand you most often?
 - Is there anything you should never have access to?
3. Recommend at most 3 connectors to add now, ranked by how much manual work each removes. For each one give me: what it unlocks in plain English, and the one-line setup step
4. Skip anything that needs an approval I have not given

Do not connect anything yet. This is the map; I pick what we wire up.

LEVEL 2 · FIND AND TRY A CONNECTOR FROM GITHUB

Find me an MCP server for [app, e.g. Premiere Pro] and get it running here.

An MCP server is a small connector program that gives you tools for an app.

1. Search GitHub for MCP servers for that app. Shortlist up to 3, preferring ones that are recently updated, have real users, and clearly list their tools
2. Before installing anything, show me the shortlist with: the tools each one exposes, how it installs, and any credentials it needs. Then ask me in one batch: which one to try, where credentials should come from, and whether write actions are allowed or read-only for now
3. Install my pick, wire it into this setup, and restart what needs restarting
4. Prove it works: run one safe, read-only tool call and show me the result
5. If it fails, say exactly why and move to the next one on the shortlist

Never send my credentials anywhere except the app's own login flow.

LEVEL 3 · BUILD YOUR OWN MICRO-APP

Build me a micro-app for a job I do every week: [e.g. "a page where I paste a YouTube link and get back my title and thumbnail checklist"]

Ask me first, in one batch:

1. What exact inputs go in, and what should come out?
2. Where will I use it: a browser page, a Chrome extension, or a command in the terminal?
3. Should it remember anything between uses?

Then:

1. Build the smallest version that does the whole job: one HTML file, one script, or one CLI command. No frameworks unless truly needed
2. Wire it to real data, not placeholder text
3. Put it where my other tools live, and give me the exact way to open or run it
4. Walk me through one real use, end to end, before calling it done

If the job turns out to need two tools, tell me and build the more valuable one first.

CLI VS API VS MCP, IN ONE BREATH

A **CLI** is a command-line tool your agent runs directly - usually the fastest and cheapest. An **API** is the app's raw web interface - flexible, needs keys. An **MCP server** is a connector that hands your agent a menu of ready-made tools for the app. Prefer whichever exists, is maintained, and does the one job you need - and when none exists, level 3 is the answer.

The visual command centre

With ARMS in place, put a face on it: one local page that is the front door to everything. This layer is only 20 to 30% of the value – but it is the part you see all day, it makes the system feel real, and if you serve clients it is a service you can restyle and sell. These principles matter more than any specific layout:

1. **One page, one address.** It lives at the same local address every day and survives a restart (a tiny static server kept alive by a process manager). If it is not always there, you will stop opening it.
2. **Show, do not store.** The page reads your workspace and your apps – it is a window, never the place where truth lives. Artifacts stay files; the centre just finds them.
3. **Every widget earns its place.** Glanceable summaries (calendar, inbox, your numbers) and one-click actions (micro-apps, headless skill buttons, routine board). If you stop looking at a widget, delete it.
4. **Artifacts, one click away.** A section that lists what your agents made recently – every report, draft, and page – so review means opening the centre, not hunting folders.
5. **The second brain is the centrepiece.** Link your visual memory map (Memory level 3) from the middle of the page. Finding and showing your files becomes one click.
6. **Start with three widgets.** Ship small, then grow one prompt at a time. Because your agent builds and edits the page, every new widget is a conversation, not a project.
7. **Restyle freely.** The same skeleton wears any brand – yours, or a client's. The look is a skin; ARMS underneath is the product.

WANT IT PRE-BUILT?

A ready-made version of this command centre and the visual second brain is available inside the RoboNuggets community, along with the lessons on how it is put together.

BUILD YOUR COMMAND CENTRE V1

Build me the first version of my visual command centre: one local HTML page that is the front door to my agent workspace.

Ask me first, in one batch:

1. What are the 4-6 things I check or open every day?
2. What files do my agents produce that I want one click away?
3. What look do I want: match my brand, or clean and minimal?

Then build one self-contained HTML file that shows:

- My recent agent-made artifacts (scan a folder like output/artifacts/), each opening on click
- Shortcut tiles for my daily micro-apps and key folders
- A slot for the numbers I care about: fill with real data where you can read it, and mark placeholders clearly where you cannot
- Room to grow: comment the file so future widgets are easy to add

Serve it locally so it always opens at the same address (for example with a tiny static server kept alive by pm2), and tell me how to make that survive a restart.

When it is running, give me the address and a one-line summary of every section on the page.

THEN WIRE IN THE REST

From here, each upgrade is one prompt: add a headless skill button (Skills level 3, page 5), link your second brain in the centre (Memory level 3, page 7), and show your routine board (Routines, page 8). The order does not matter - add whichever gap you feel first.

The 7-day **setup** plan

Same order as the lesson: bottom-up. Skills first because they compound into everything else, then memory, then the layers that run without you. One part a day, small on purpose – a system you finish beats a bigger one you abandon.

Day 1	Skills. Run the Level 1 prompt (page 4) on your most repeated task.
Day 2	Skills again. Split one thick skill into a file set (page 4), or wire one skill to a button (page 5).
Day 3	Memory. Run the router-files prompt (page 7). Master router + one area index.
Day 4	Routines. Run the local-routine prompt (page 9) on the most annoying weekly chore.
Day 5	Applications. Run the connector audit (page 11); wire up the top pick.
Day 6	Command centre. Run the v1 prompt (page 14). Three widgets, no more.
Day 7	Test day. Run everything once. Fix misses by editing files – skills and routers – not by re-explaining in chat.

KEEP IT ALIVE

Fifteen minutes a week: prune stale routers, tighten one skill file, check your routine outputs actually landed. And keep the do-it-twice rule running – every repeated prompt becomes a skill, every new area gets its router line. The system compounds only while the files stay true.

Where this **goes** next

The easy part is the setup: four parts, a dozen prompts, one week. The skill that pays is **operating** it - noticing which task deserves a skill, which skill deserves a button, which routine deserves its own machine. That judgment comes from reps, and reps come faster around people building the same thing.

Inside the RoboNuggets community, members run the pre-built version of this OS - the command centre and the visual second brain - and learn to build and sell AI systems like it for real businesses, mastering Claude Code and Claude Design along the way.

skool.com/robonuggets

The ARMS checklist

Four parts, one test each. When all four boxes tick, you have an operating system - and the command centre makes it yours.

S

Skills

Your most repeated task runs from one named command - and one skill fires from a button, not a chat.

☐**M**

Memory

A brand-new session finds the right file through your routers on the first hop.

☐**R**

Routines

At least one piece of work ships on schedule with your computer off.

☐**A**

Applications

Your daily apps are reachable by your agent, and one tool you built yourself is part of your week.

☐

Four parts. Three levels. One system. Yours.